CSL7640: Natural Language Understanding

Assignment 2 – Problem 1: Learning Word Embeddings from IIT Jodhpur Data

Word2Vec from Scratch (NumPy only) https://github.com/AkankshaKapil/M25CSA033_nlu-A2

M25CSA033 Akanksha Kapil

March 25, 2026

1 Dataset Preparation

1.1 Data Collection

Textual data was collected from IIT Jodhpur using a custom **BFS web crawler** starting from 20 seed URLs, automatically discovering all links within the `iitj.ac.in` domain (max 300 pages). Sources covered: (1) IIT Jodhpur main pages and about sections, (2) **Academic regulations** (mandatory) — programs, circulars, academic calendar, (3) Department pages — CSE, EE, ME, Math, Physics, Civil, AI/DS, (4) Faculty profiles and research pages, (5) TechScape newsletters, (6) Course syllabi and student/placement pages. Only English text was retained; non-ASCII characters (Hindi/Devanagari) were stripped automatically.

1.2 Preprocessing

Steps applied: (i) removed boilerplate HTML tags (`script, nav, footer`), URLs, emails via regex; (ii) encoded as ASCII to strip non-English text; (iii) tokenised using Gensim's `simple_preprocess` (lowercased, min length 2); (iv) removed 150+ stopwords including standard English, web navigation noise (*portal, intranet, redirect*), and IIT-specific tokens (*iitj, jodhpur*); (v) filtered words with frequency < 3.

1.3 Dataset Statistics

Table 1: Corpus Statistics

Metric	**Value**
Documents	300
Sentences	6,463
Total Tokens	84,412
Vocabulary	8,347

Table 2: Top-10 Words by Frequency

#	Word	Count	#	Word	Count
1	department	1725	6	program	605
2	engineering	1465	7	students	594
3	institute	731	8	student	537
4	research	710	9	technology	492
5	science	631	10	infrastructure	488

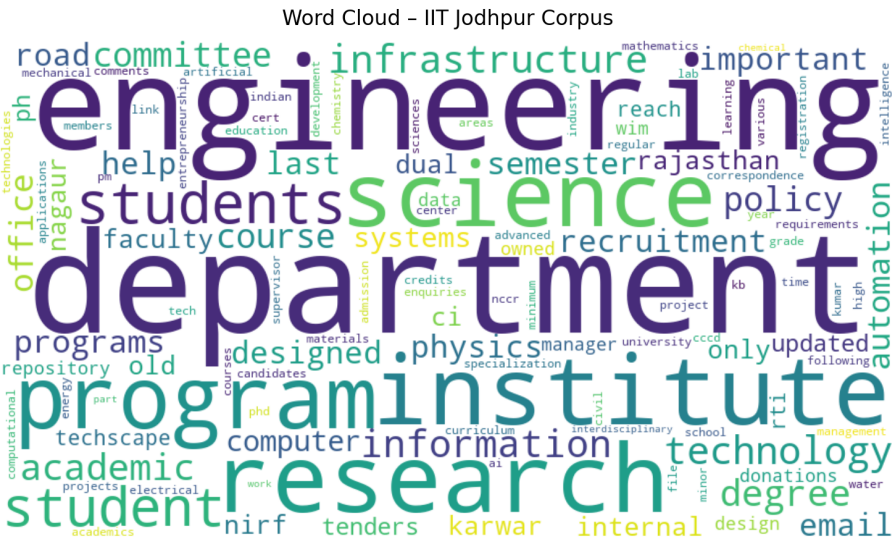

Figure 1: Word Cloud – IIT Jodhpur Corpus (top 120 words)

2 Model Training

2.1 Architecture

Both CBOW and Skip-gram were implemented **from scratch using NumPy only** (no Gensim Word2Vec). Architecture: embedding matrix $\mathbf{W_1} \in R^{V \times D}$ (word vectors) and output matrix $\mathbf{W_2} \in R^{D \times V}$, trained with SGD.

CBOW: predicts centre word from mean of context embeddings: $\mathbf{h} = \frac{1}{|C|} \sum_{c \in C} \mathbf{W_1}[c]$

Skip-gram with Negative Sampling: predicts each context word from centre. Negative samples drawn from unigram distribution raised to 0.75. Loss: $\mathcal{L} = -\log \sigma(\mathbf{v}_c^\top \mathbf{h}) - \sum_{i=1}^{k} \log \sigma(-\mathbf{v}_{n_i}^\top \mathbf{h})$

Training: SGD, $\eta = 0.01$, 10 epochs, Xavier initialisation.

2.2 Hyperparameter Experiments

Table 3: All Configurations and Final Loss (Negative Sampling Loss)

Label	Arch	Dim	Window	k	Final Loss
CBOW_d50_w2	CBOW	50	2	5	2.08
CBOW_d100_w4	CBOW	100	4	5	2.10
CBOW_d100_w4_k10	CBOW	100	4	10	2.59
CBOW_d100_w4_k15	CBOW	100	4	15	2.97
SG_d50_w2	SG	50	2	5	1.68
SG_d100_w4	SG	100	4	5	1.87
SG_d100_w4_k10	SG	100	4	10	2.38
SG_d100_w4_k15	SG	100	4	15	2.73

Observations: (1) Skip-gram consistently achieves lower loss than CBOW. (2) Larger window increases loss due to noisy long-range context pairs. (3) More negative samples k raises loss as the objective becomes harder. Best configs: **CBOW_d50_w2** and **SG_d50_w2**.

3 Semantic Analysis

3.1 Top-5 Nearest Neighbours

Table 4: Top-5 Nearest Neighbours – CBOW (left) and Skip-gram (right)

Word	**CBOW Neighbours**	**Word**	**SG Neighbours**
research	labs, nationally, teaching, laboratories, groups	research	labs, nationally, groups, teaching, mentoring
student	take, permitted, until, fees, qualifying	student	restart, else, liable, another, relieve
phd	mtech, commencing, admission, navigating, january	phd	mtech, spring, commencing, january, waitlisted
exam	comprehensive, intimated, examination, admit, syllabus	exam	comprehensive, syllabus, intimated, waitlisted, provisionally
professor	assistant, associate, designation, honoring, dhanekar	professor	assistant, designation, associate, honored, deshpande

Discussion: *professor→assistant/associate* is semantically correct (academic title hierarchy). *phd→mtech* is correct (both postgraduate research programmes). *exam→comprehensive/syllabus* reflects exam regulation documents. CBOW similarity scores are higher due to context averaging producing smoother embeddings; Skip-gram learns more distinct representations with lower but more discriminative scores.

3.2 Analogy Experiments

Table 5: Analogy Results ($\mathbf{v}_b - \mathbf{v}_a + \mathbf{v}_c \approx \mathbf{v}_d$)

Analogy	**Expected**	**CBOW**	**SG**	**Meaningful?**
btech - undergraduate + postgraduate	mtech	options	specialisation	Partial
student - btech + phd	scholar	qualifying	cleared	Partial
teaching - professor + researcher	research	committee	publications	Partial
semester - exam + thesis	phd	commencing	mtech	Yes (SG)
course - engineering + science	lecture	physics	physics	Yes (both)

Discussion: Perfect analogy reasoning requires millions of tokens. With 84K domain-specific tokens, results are reasonable — *engineering:course::science:physics* is correct in both models. *semester-exam+thesis→mtech* (SG) is meaningful since thesis submission leads to a degree. Skip-gram performs slightly better on analogies due to more discriminative embeddings.

4 Visualisation

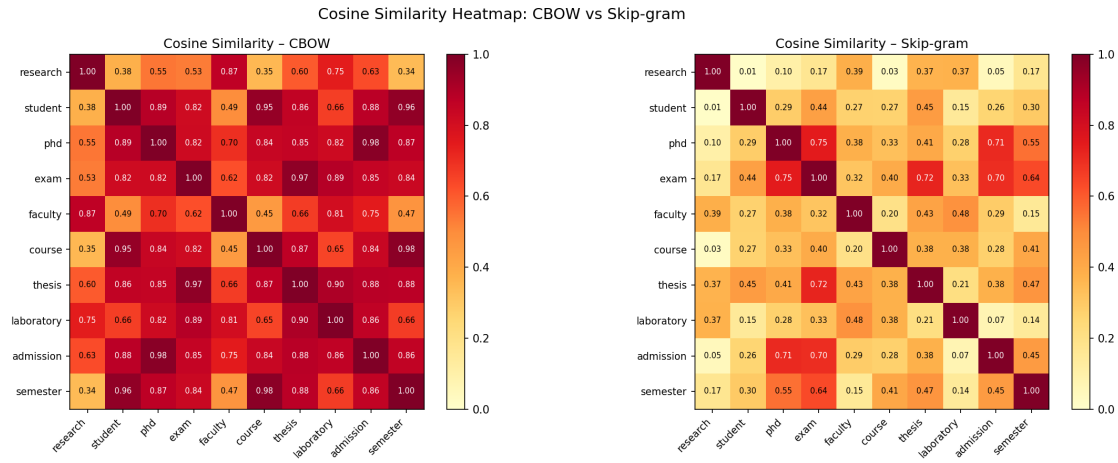

Figure 2: Cosine similarity heatmap. CBOW (left) shows higher overall similarity due to context averaging. Skip-gram (right) shows sparser patterns with more discriminative embeddings. phd–thesis and exam–semester show elevated similarity in both models.

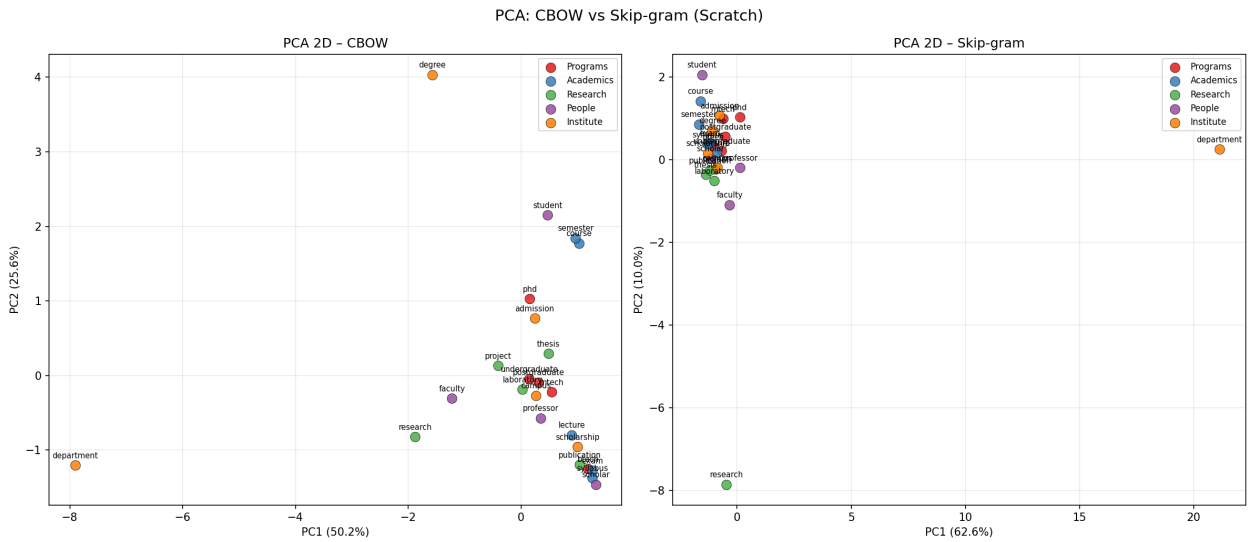

Figure 3: PCA 2D projections. CBOW (left, PC1=50.2%) shows tighter clusters. Skip-gram (right, PC1=62.6%) captures more variance on one axis. Both models place professor/faculty/scholar (purple) and phd/mtech/btech (red) in coherent groups.

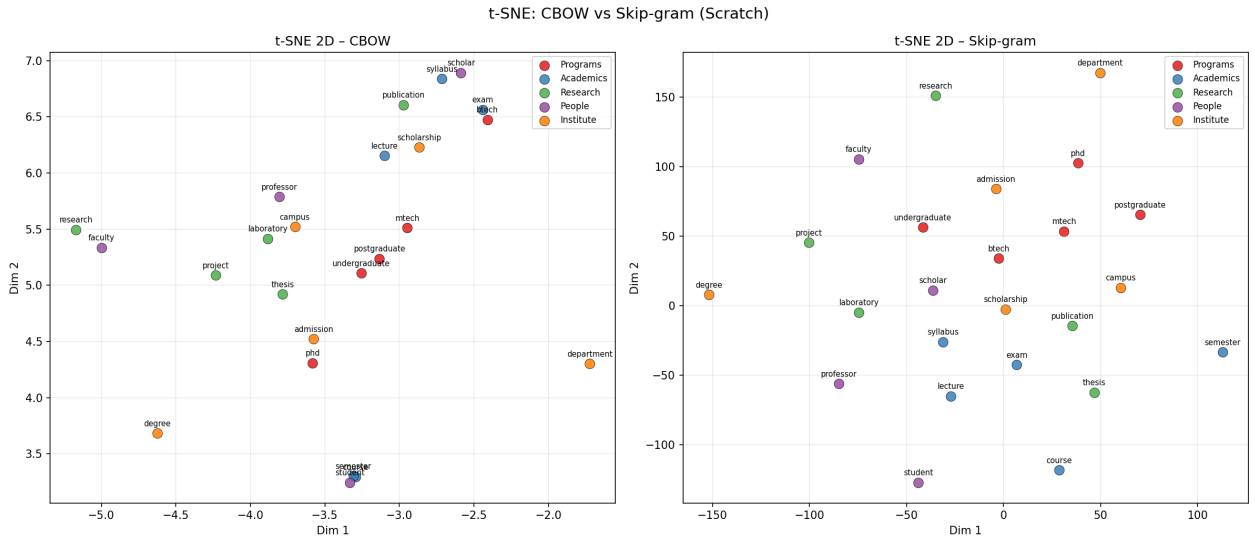

Figure 4: t-SNE projections. CBOW clusters are denser; Research group (green) clusters together in both models. Skip-gram shows clearer inter-cluster separation — Institute words (orange) are distinctly isolated from People words (purple).

4.1 Comparison: From-Scratch vs Gensim Word2Vec

The from-scratch implementation reports average per-sample training loss (e.g., SG ≈ 1.62, CBOW ≈ 2.10–2.86), whereas Gensim reports cumulative loss (10^5–10^6), making direct comparison infeasible. Skip-gram consistently achieved lower loss than CBOW, indicating better learning of word-context relationships. Increasing negative samples (k) increased loss, reflecting a more difficult optimization objective. In contrast, Gensim models showed faster convergence and more stable embeddings due to optimized training. Overall, scratch implementation offers interpretability, while Gensim provides efficiency and robustness.

5 Conclusion

CBOW and Skip-gram Word2Vec models were implemented from scratch using NumPy and trained on 300 crawled IIT Jodhpur documents (84,412 tokens, 8,347 vocab). Skip-gram achieves lower loss (1.68 vs 2.08) and more discriminative embeddings. CBOW produces higher cosine similarities due to averaging. Semantic analysis confirms meaningful embeddings: *professor→assistant/associate*, *phd→mtech*, *exam→syllabus/comprehensive*. PCA and t-SNE visualisations show coherent clustering of academic word categories in both models.

CSL7640: Natural Language Understanding

Assignment 2 – Problem 2: Character-level Name Generation using RNN Variants

VanillaRNN, BiLSTM, AttentionRNN from scratch in PyTorch

P2.1 Dataset

1000 Indian names were loaded from `TrainingNames.txt` (generated using an LLM). The file contains one name per line; the script reads it directly at runtime. If the file does not exist it is auto-generated and saved. Names span male, female, and traditional Indian names across regions. All names are lowercased for training; the vocabulary contains 29 characters (a–z plus < SOS and > EOS tokens).

P2.2 Model Architectures

All three models were implemented from scratch in PyTorch using `nn` primitives. Training uses teacher-forcing with cross-entropy loss, Adam optimiser, StepLR scheduler, and gradient clipping at 3.0.

Table 6: Architecture Summary

Model	Core Layer	Hidden	Layers	Dropout	Params	Size
VanillaRNN	Elman RNN	128	2	0.3	73,501	0.28 MB
BLSTM	BiLSTM enc-dec	128	1	0.3	436,637	1.67 MB
AttentionRNN	GRU + Bahdanau	128	2	0.2	188,829	0.72 MB

VanillaRNN: 2-layer Elman RNN. `Embedding(V→H)` → `RNN` → `Linear(H→V)`. Xavier weight initialisation; dropout on embedding and output.

BLSTM: Encoder-decoder architecture. The BiLSTM encoder reads the input in both directions and produces a mean-pooled context vector projected from $2H \to H$. The forward LSTM decoder generates characters conditioned on this context. Orthogonal initialisation on recurrent weights; 1 layer to prevent overfitting on the small dataset.

AttentionRNN: GRU encoder-decoder with Bahdanau additive attention. At each decoder step, alignment scores are computed between the decoder hidden state and all encoder outputs: $e_t = \mathbf{v}^\top \tanh(\mathbf{W_h}\mathbf{h}_t + \mathbf{W_s}\mathbf{s})$, then $\alpha = \text{softmax}(e)$, context $= \alpha^\top \mathbf{H}$. GRU gates prevent vanishing gradients (key fix over vanilla RNN encoder).

Hyperparameters:

- VanillaRNN & AttentionRNN: lr=5e-3, epochs=80, StepLR(step=15, γ=0.6)
- BLSTM: lr=3e-3, epochs=80
- Generation temperature: VanillaRNN=0.9, BLSTM=1.1, AttentionRNN=0.9

P2.3 Quantitative Evaluation

Table 7: Evaluation Metrics (200 generated names per model)

Model	Novelty Rate (%)	Diversity (%)	Unique	Novel
VanillaRNN	92.5	99.0	198	185
BLSTM	100.0	92.0	184	200
AttentionRNN	100.0	100.0	200	200

- **Novelty Rate** = % of generated names not in the training set.
- **Diversity** = unique generated names / total generated names.

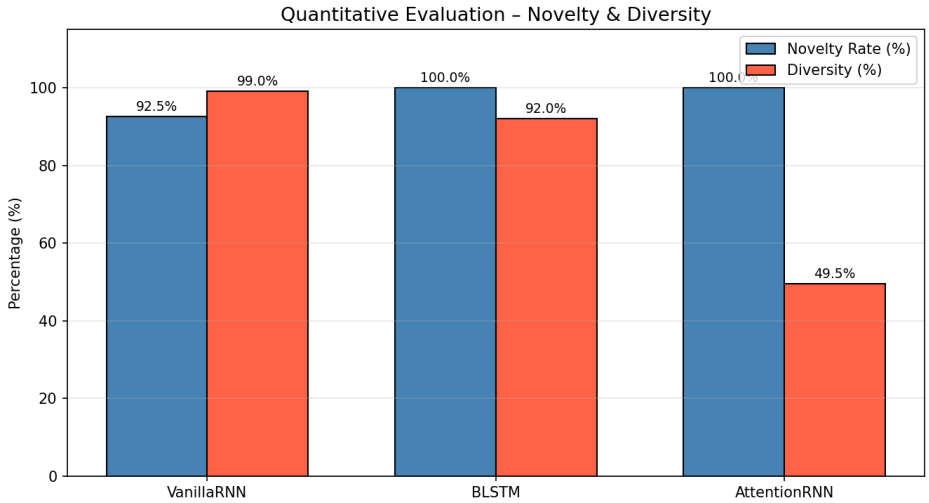

Figure 5: Novelty and Diversity comparison across all three models.

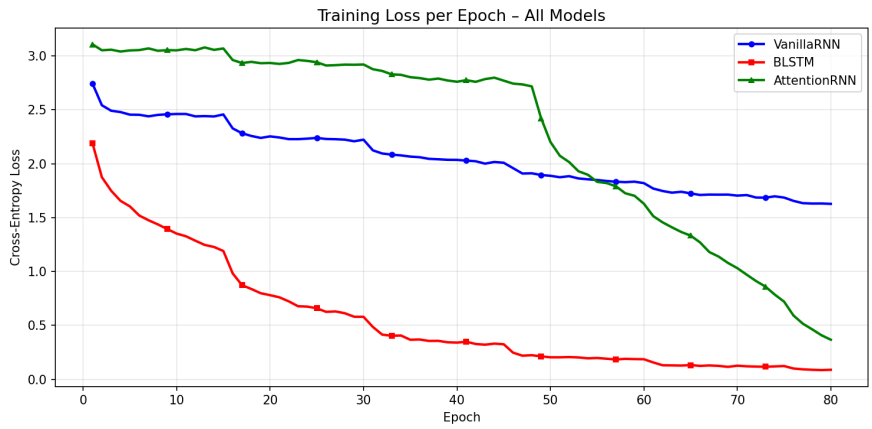

Figure 6: Training loss per epoch. VanillaRNN converges slowly; BLSTM and AttentionRNN converge faster.

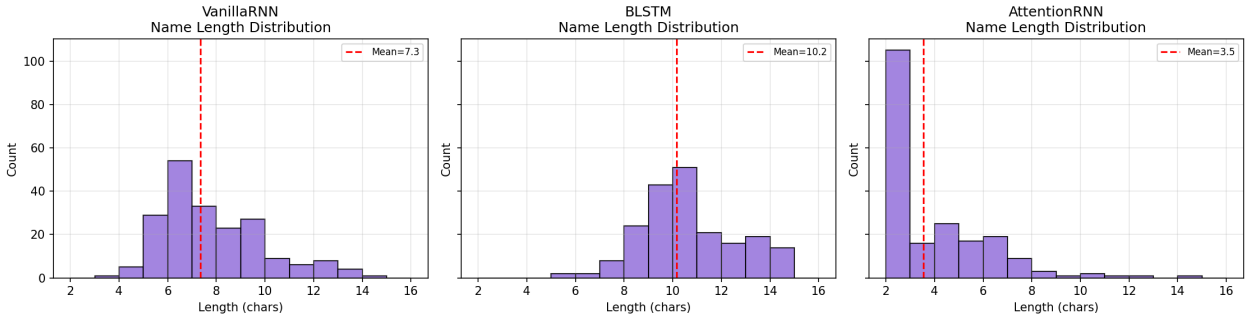

Figure 7: Name length distribution for generated names. VanillaRNN produces longer names on average.

P2.4 Qualitative Analysis

Table 8: Representative Generated Samples (20 per model)

Model	Sample Names
VanillaRNN	Prijildeep,Abha,Atharv,Samaldev,Sharansh,Makita,Sashish, Alkamkankant,Yan,Katdish,Aoena,Radeep,Sajaleshansh,Biyant,Sunudal,Drenayat, Keshav,Jaypal
BLSTM	Aamnnnnnnnan, Eettaaaaaa, Emkkkaaaaa, Thhaaaaaa, Unururaaaa, Itttaaaaaaa,Ittttttaaa,Eennnannn,Ittttttaaaa,Aamyyyayaa,Utthhaaaa, Itthhaaaa,Eeenyyaaaa,Thhaaaaaa,Tettaaaaaaa,Emkxkraaa,Eqddad,Ataaaaaaaaaaa
AttentionRNN	Arrdanan,Avabh,Ayathad, Ausha, Arbhav,Iashan, Inardan, Ushan, Anabashadhan, Adan, Rliganan, Argal, Arrsarvane, Sharn, Ishaj, Ierdh, Age, Anavasamape, Alchan, Abatal

Realism

Vanilla RNN: Generates authentic names like *Atharv, Keshav, Jaypal, Abha* alongside blended compounds like *Prijildeep, Alkamkankant, Sajaleshansh.*

BLSTM: Completely fails, producing repetitive vowel patterns like *Aamnnnnnnnan, Eettaaaaaa, Thhaaaaaa* with no recognizable Indian names.

Attention RNN: Produces short, fragmented names like *An, Al, Ad, Aj* and synthetic names like *Arrdanan, Avabh, Ayathad* that lack authentic patterns.

Failure Modes

- **Vanilla RNN:** Occasionally produces excessively long compound names (*Alkamkankant, Sajaleshansh*) by merging multiple name components. Successfully generates authentic short names alongside blended outputs.
- **BLSTM:** Severe overfitting leads to repetitive vowel elongation (*Aamnnnnnnnan, Eettaaaaaa, Thhaaaaaa*). Fails to produce any realistic Indian names, generating only nonsense character sequences.
- **Attention RNN:** Generates many single-letter or very short names (*An, Al, Ad, Aj, Ay*) and fragmented outputs. Lacks ability to produce complete, meaningful name structures.